NaN (Need a Name)

1 G

1.1 Plantilla

```
#define FIN ios::sync_with_stdio(0);cin.tie(0);cout.tie(0)
```

1.2 Compilar

```
# Llenar lo siguiente en el build de geany

# Compile
g++ -std=c++17 -DLOCAL -g -O2 -Wconversion -Wshadow -Wall -Wextra -D_GLIBCXX_DEBUG -c %f

# Build
g++ -std=c++17 -DLOCAL -g -O2 -Wconversion -Wshadow -Wall -Wextra -D_GLIBCXX_DEBUG -o %e %f
```

2 Data Structures

2.1 Compresion De Coordenadas

```
struct compresion {
    vector<int> todos;
    compresion(vector<int> v) {
        todos = v; sort(all(todos));
        todos.erase(unique(all(todos)),todos.end());
    }
    int obtener(int x) { return (int)(lower_bound(all(todos),x)-todos.begin()); }
};
```

2.2 Dsu

```
struct DSU {
    vi link, sz;
    DSU(int tam) {
        link.resize(tam+5), sz.resize(tam+5);
        forn(i, tam+5) link[i] = i, sz[i] = 1;
    }
    ll find(ll x){ return link[x] = (link[x] == x ? x : find(link[x])); }
    bool same(ll a, ll b) { return find(a) == find(b); }
    void join(ll a, ll b) {
        a = find(a), b = find(b);
        if(a == b) return;
        if(sz[a] < sz[b]) swap(a,b);
        sz[a] += sz[b];
        link[b] = a;
    }
};
```

2.3 Implicit Treap

```
typedef struct item *pitem;
struct item {
    ll key, prior, cont, mini, sum;
    bool rev; // (parameters for lazy prop)
    pitem l, r;
    item(ll key):
        key(key),prior(rand()),cont(1),l(NULL),r(NULL),rev(0) {}
};
void push(pitem it) { //Lazy for reverse array
    if(it && it->rev) {
        swap(it->l,it->r);
        if(it->l)it->l->rev ^= true;
        if(it->r)it->r->rev ^= true;
        it->rev = false;
    }
}
int cont(pitem it) {return it ? it->cont : 0;}
ll mini(pitem it) {return it ? it->mini : 1<<30;}
ll sum(pitem it) {return it ? it->sum : 0LL;}
void upd_cont(pitem it){
    if(it) {
        it->cont = cont(it->l) + cont(it->r) + 1;
        it->mini = min(it->key,min(mini(it->l),mini(it->r)));
        it->sum = it->key + sum(it->l) + sum(it->r);
    }
}
void merge(pitem &t, pitem l, pitem r) { //Merge l and r, new root t
    push(l); push(r);
    if(!l || !r) t = l ? l : r;
    else if(l->prior > r->prior) merge(l->r,l->r,r), t=l;
    else merge(r->l,l,r->l), t = r;
    upd_cont(t);
}
void split(pitem t, pitem& l, pitem& r, int sz){ //sz:desired size of l
    if(!t) { l = r = 0; return;}
    push(t);
    if(sz <= cont(t->l)) split(t->l,l,t->l,sz), r = t;
    else split(t->r,t->r,r,sz-1-cont(t->l)), l = t;
    upd_cont(t);
}
int find_min(pitem t) { //Devuelve la posicion del minimo
    push(t);
    if(t->mini == t->key) return cont(t->l);
    else if(t->l && t->l->mini == t->mini) return find_min(t->l);
    else return cont(t->l)+1+find_min(t->r);
}
void set_treap(pitem &root, int p, ll val) {
    pitem aux1 = NULL, aux2 = NULL;
    split(root,root,aux2,p+1); split(root,root,aux1,p);
    aux1->key = val;
    merge(root,root,aux1); merge(root,root,aux2);
}
ll get_sum(pitem &root, int l, int r) {
    pitem aux1 = NULL, aux2 = NULL;
    split(root,root,aux2,r+1); split(root,root,aux1,l);
    ll ans = aux1->mini; // aux1->mini for minimum
    merge(root,root,aux1); merge(root,root,aux2);
```

```
    return ans;
}
void DFS(pitem t) { //Good for debug
    DBG(t->key);
    if(t->l != NULL) DBG(t->l->key); if(t->r != NULL) DBG(t->r->key);
    RAYA;
    if(t->l != NULL) DFS(t->l); if(t->r != NULL) DFS(t->r);
}
```

2.4 Indexed Set

```
using namespace __gnu_pbds;
typedef tree<ll,null_type,less<ll>,rb_tree_tag, tree_order_statistics_node_update> indexed_set;
indexed_set s; s.insert(2); s.insert(3); s.insert(5);
auto x=s.find_by_order(2); // retorna el valor en la posicion 2 (5)
int pos=int(s.order_of_key(3)); // retorna la posicion que se encuentra el 3 (1)
```

2.5 Lazy-Propagation

```
typedef long long tipo;
const int NEUT = 0; // REMINDER !!!
struct node {
  tipo ans, l, r, lazy = 0;
  bool upd = false;
  node() { ans = lazy = 0; upd = false; l = r = -1; } // REMINDER !!! SET NEUT
  node(tipo val, int pos) : ans(val), l(pos), r(pos) {} // Set node
  void set_lazy(tipo x) { lazy += x; upd = true; }
};
struct segtree_lazy {
  #define l(x) int(x<<1)
  #define r(x) int(x<<1|1)
  vector <node> t; int tam;
  node op(node a, node b) {
    node aux; aux.ans = a.ans + b.ans; // Operacion de query
    aux.l = a.l; aux.r = b.r;
    return aux;
  }
  void node_update(node &cur) {
    cur.ans += cur.lazy * (cur.r-cur.l+1); // Operacion update
  }
  void reset_lazy(node &cur) {
    cur.lazy = 0; cur.upd = false; //Poner el neutro del update
  }
  void push(int p) {
    node &cur = t[p];
    if(cur.upd == true) {
      node_update(cur);
      if(cur.l < cur.r) {
        t[l(p)].set_lazy(cur.lazy);
        t[r(p)].set_lazy(cur.lazy);
      }
      reset_lazy(cur);
    }
  }
  node query(int l, int r, int p = 1) {
    push(p); node &cur = t[p];
    if(l > cur.r || r < cur.l) return node(); // Return NEUT
    if(l <= cur.l && cur.r <= r) return cur;
    return op(query(l,r,l(p)),query(l,r,r(p)));
  }
  void update(int l, int r, tipo val, int p = 1) { // root at p = 1
    push(p); node &cur = t[p];
    if(l > cur.r || r < cur.l) return;
    if(l <= cur.l && cur.r <= r) {
      cur.set_lazy(val); push(p); return;
    }
    update(l, r, val, l(p)); update(l, r, val, r(p));
    cur = op(t[l(p)], t[r(p)]);
  }
  void build(vector <tipo> v, int n) { // iterative build
    tam = sizeof(int) * 8 - __builtin_clz(n); tam = 1<<tam;
    t.resize(2*tam); v.resize(tam);
    forn(i,tam) t[tam+i] = node(v[i],i);
    for(int i = tam - 1; i > 0; i--) t[i] = op(t[l(i)],t[r(i)]);
  }
};
```

2.6 Mo Dquery

```
struct query{
    int l, r, ind;
    bool operator <(query q) const {
        return r < q.r;
    }
};
struct mo {
    const int block = 448; // sqrt(MAXN)
    vector<vector<query>> q;
    vector<ll> ans, cont;
    mo(){
        ans.resize(MAXN), cont.resize(MAXN);
        q.resize(MAXN/block+5);
    }
    void read_queries(int m) { // m --> number of queries
        ans.resize(m,0);
        forn(i, m) {
            int l, r; cin >> l >> r;
            l--; r--; // For 0-indexed
            q[l / block].pb({l,r,(int)i});
        }
    }
    void add(ll &sum, ll val) {
        if(cont[val] == 0) sum++;
        cont[val]++;
    }
    void erase(ll &sum, ll val) {
        if(cont[val] == 1) sum--;
        cont[val]--;
    }
    void get_mo(vector<ll> &v) {
        ll sum = 0;
```

```cpp
        forn(i,q.size()) {
            sort(all(q[i]));
            int l, r; l = r = block * i;
            for(query u : q[i]) { // Solve for [l,r]
                while(r <= u.r) add(sum,v[r]), r++;
                while(l <= u.l) erase(sum,v[l]), l++;
                while(l > u.l) l--, add(sum,v[l]);
                ans[u.ind] = sum;
            }
            while(l < r) erase(sum,v[l]), l++;
        }
    }
};
```

2.7 Persistent

```cpp
struct Vertex {
    Vertex *l, *r;
    ll sum;
    Vertex(ll val) : l(nullptr), r(nullptr), sum(val) {}
    Vertex(Vertex *l, Vertex *r) : l(l), r(r), sum(0) {
        if (l) sum += l->sum;
        if (r) sum += r->sum;
    }
};
Vertex* build(ll a[], ll tl, ll tr) {
    if (tl == tr)
        return new Vertex(a[tl]);
    ll tm = (tl + tr) / 2;
    return new Vertex(build(a, tl, tm), build(a, tm+1, tr));
}
ll get_sum(Vertex* v, ll tl, ll tr, ll l, ll r) {
    if (l > r)
        return 0;
    if (l == tl && tr == r)
        return v->sum;
    ll tm = (tl + tr) / 2;
    return get_sum(v->l, tl, tm, l, min(r, tm))
         + get_sum(v->r, tm+1, tr, max(l, tm+1), r);
}
Vertex* update(Vertex* v, ll tl, ll tr, ll pos, ll new_val) {
    if (tl == tr)
        return new Vertex(new_val);
    ll tm = (tl + tr) / 2;
    if (pos <= tm)
        return new Vertex(update(v->l, tl, tm, pos, new_val), v->r);
    else
        return new Vertex(v->l, update(v->r, tm+1, tr, pos, new_val));
}
vector <Vertex*> estado;
```

2.8 Segtree-Iterative

```cpp
typedef long long tipo;
struct segtree {
  vector <tipo> t; int tam;
  tipo NEUT = 0; // Neutral element of operation
  tipo op(tipo a, tipo b){ return a+b; } // Operation to make
  void build(vector<tipo>v, int n) {  // build the tree
    // root en 1, ojas en el intervalo [tam, 2*tam-1]
    tam = sizeof(int) * 8 - __builtin_clz(n); tam = 1<<tam;
    t.resize(2*tam,NEUT); forn(i, n) t[tam+i] = v[i];
    for(int i = tam - 1; i > 0; i--) t[i] = op(t[i<<1], t[i<<1|1]);
  }
  void update(int p, tipo value) {  // set value at position p
    for (t[p += tam] = value; p > 1; p >>= 1) t[p>>1] = op(t[p], t[p^1]);
    //Tener cuidado aca, podria ser que en cosas no conmutativas, para p impar: p > p^1.
  }
  tipo query(int l, int r) {  // op on interval [l, r]
    tipo res = NEUT;
    for (l += tam, r += tam; l <= r; l >>= 1, r >>= 1) {
      if (l&1) res = op(res, t[l++]);
      if (!(r&1)) res = op(res, t[r--]);
    }
    return res;
  }
};
```

3 Dp

3.1 Cht Dynamic

```cpp
// Convex Hull trick dinamico.
// O(log n) para agregar e consultar.
// Para minimo, cambiar el signo de m y b.
typedef ll tc;
const tc is_query=-(1LL<<62); // special value for query
struct Line {
    tc m,b;
    mutable multiset<Line>::iterator it,end;
    const Line* succ(multiset<Line>::iterator it) const {
        return (++it==end? NULL : &*it);}
    bool operator<(const Line& rhs) const {
        if(rhs.b!=is_query)return m<rhs.m;
        const Line *s=succ(it);
        if(!s)return 0;
        return b-s->b<(s->m-m)*rhs.m;
    }
};
struct HullDynamic : public multiset<Line> { // for maximum
    bool bad(iterator y){
        iterator z=next(y);
        if(y==begin()){
            if(z==end())return false;
            return y->m==z->m&&y->b<=z->b;
        }
```

```
        iterator x=prev(y);
        if(z==end())return y->m==x->m&&y->b<=x->b;
        return (x->b-y->b)*(z->m-y->m)>=(y->b-z->b)*(y->m-x->m);
    }
    iterator next(iterator y){return ++y;}
    iterator prev(iterator y){return --y;}
    void add(tc m, tc b){
        // m *= -1; b *= -1; --> For minumum
        iterator y=insert((Line){m,b});
        y->it=y;y->end=end();
        if(bad(y)){erase(y);return;}
        while(next(y)!=end()&&bad(next(y)))erase(next(y));
        while(y!=begin()&&bad(prev(y)))erase(prev(y));
    }
    tc eval(tc x){
        Line l=*lower_bound((Line){x,is_query});
        return l.m*x+l.b; // -1*(l.m*x + l.b) for minimum
    }
};
```

3.2 Divide And Conquer Dp Opt

```
// Divide and conquer optimization for DP
// Complexity: O(nklogn)
// Usage: solve_dac(n,k); // n --> number of elements, k --> number of groups
ll costo[MAXN][MAXN];
vector <ll> last(MAXN), dp(MAXN);
void calc_costo(int n, vector <ll> &v) {
    // Dependiendo el problema, aca dentro realizamos la funcion para
    // precalcular el costo de cada intervalo y guardarlo en 'costo'
}
void compute(int l, int r, int optl, int optr) {
    if(l > r) return;
    int med = (l+r)/2;
    pair<ll,ll> best = {INF,-1};
    for(int p = optl; p <= min(med,optr); p++) {
        best = min(best,{last[p] + costo[p+1][med],p});
    }
    dp[med] = best.first;
    int opt = best.second;
    compute(l,med-1,optl,opt);
    compute(med+1,r,opt,optr);
}
ll solve_dac(int n, int k) { // divide and conquer optimization
    for(int i = 0; i < n; i++) last[i] = costo[0][i];
    for(int i = 2; i <= k; i++) {
        fill(all(dp),INF);
        compute(0,n-1,0,n-1);
        last = dp;
    }
    return dp[n-1];
}
```

3.3 Knapsack Weighted

```
vector<bool> knapsack_weighted(vector <int> v, int n) {
  int sum = accumulate(all(v),0LL);
  vector <bool> dp(sum+1,false);
  dp[0] = true;
  sort(all(v));
  vector <pair<int,int>> num; // (number,#counter)
  int count = 1, last = v[0];
  forr(i,1,n) {
    if(v[i] == last) count++;
    else {
      num.pb({last,count});
      last = v[i];
      count = 1;
    }
  }
  num.pb({last,count});
  for(auto [x,c] : num) {
    vector <bool> next_dp = dp;
    forn(i,x) {
      int can = 0;
      for(int j = i; j <= sum; j += x) {
        next_dp[j] = next_dp[j] | (can>0);
        if(dp[j]) can++;
        if(j - c*x >= 0 and dp[j-c*x]) can--;
      }
    }
    dp = next_dp;
  }
  return dp;
}
```

3.4 Knuth Division Dp Opt

```
const ll INF = (ll)(1e18+10);
const int MAXN = 5005;
vector <ll> prefix;
// Sea Pos(i,j) la posicion del corte que optimiza la dp, intervalo [i,j]
// Se puede aplicar optimizacion de Knuth si se cumple que:
// Pos(i,j-1) <= pos(i,j) <= pos(i+1,j)
// Objetivo: llevar un array a elementos individuales mediante cortes.
ll suma(int i, int j) {
    return prefix[j+1] - prefix[i]; // Suma de los elementos del inter
}
ll solve_Knuth(int n) {
    vector <ll> pos(n+1);
    vector <vector <ll> > dp(MAXN,vector<ll>(MAXN,INF));
    for(int i = 0; i < n; i++) dp[i][i] = 0, pos[i] = i;
    for(int len = 1; len < n; len++) {
        vector <ll> aux(n+1,0);
        for(int i = 0; i < n - len; i++) {
            for(int k = pos[i]; k <= pos[i+1]; k++) {
                ll cost = suma(i,i+len) + dp[i][k] + dp[k+1][i+len];
                if(cost < dp[i][i+len]) {
                    dp[i][i+len] = cost; aux[i] = k;
```

```
                }
            }
        }
        pos = aux;
    }
    return dp[0][n-1];
}
```

3.5 Lis

```
ll lis(vi &a, int strict = 0){
  vi temp; temp.pb(a[0]);
  forr(i, 1, SZ(a)){
    ll x = a[i];
    if(x >= temp.back()+strict) temp.pb(x);
    else {  auto it = upper_bound(all(temp), x-strict); *it = x; } }
  return SZ(temp); }
```

4 Geometry

4.1 Angular Sort

```
// Angular sort
punto origin = {0LL,0LL};
bool AngularSort(const punto &A, const punto &B) {
  bool aorig = (A < origin); // semiplano izquierdo o derecho
  bool borig = (B < origin); // semiplano izquierdo o derecho
  if (aorig != borig) { return aorig > borig; } // primero sem izq
  return (A^B) > 0; // desempate por producto vectorial si mismo sem
}
```

4.2 Calipers

```
// Devuelve la distancia entre los puntos mas lejanos en O(N)
// Cuando se usa con nuestra convex_hull, hacer reverse porque vienen CW
ld callipers(vector<punto> p, int n){
    ld r=0;     // prereq: Convex, ccw, NO COLLINEAR POINTS
    for(int i=0,j=n<2?0:1;i<j;++i){
        for(;;j=(j+1)%n){
            r=max(r,(p[i]-p[j]).mod());
            if((p[(i+1)%n]-p[i])^(p[(j+1)%n]-p[j])<=EPS)break;
        }
    }
    return r;
}
```

4.3 Closest Points Monogon

```
const ll INF = (ll)(1e15+10); // (1e18+10)
template<typename T>
using minpq = priority_queue<T, vector<T>, greater<T>>;
struct punto {
    ll x, y;
    punto (ll x = 0, ll y = 0) : x(x), y(y) {}
    punto operator -(const punto &p) const {
        return punto(x - p.x, y - p.y); }
    ll len2() const { return x * x + y * y; }
    bool const operator <(const punto &p) const {
        return (x != p.x) ? x < p.x : y < p.y;
    }
};
istream& operator>>(istream &is, punto &p) {
    return is >> p.x >> p.y;
}
ll closest(vector<punto> &p, int l, int r) { // Init l = 0 and r = p.size()-1
    if(l == r) return INF;
    int m = (l + r) / 2;
    ll mid = p[m].x;
    ll d = min(closest(p, l, m), closest(p, m + 1, r));
    vector<punto> A, B, C;
    forr(i, l, m + 1) {
        ll r = mid - p[i].x;
        if(r * r <= d) A.push_back(p[i]);
    }
    forr(i, m + 1, r + 1) {
        ll r = mid - p[i].x;
        if(r * r <= d) B.push_back(p[i]);
    }
    auto cmpy = [&](punto a, punto b) { return a.y < b.y; };
    merge(all(A), all(B), back_inserter(C), cmpy);
    forr(i, 0, C.size()) {
        int j = i + 1;
        while(j < C.size() && (C[i] - C[j]).y * (C[i] - C[j]).y <= d) {
            d = min(d, (C[i] - C[j]).len2());
            j++;
        }
    }
    inplace_merge(p.begin() + l, p.begin() + m + 1, p.begin() + r + 1, cmpy);
    return d;
}
void sorting(vector <punto> &p) { sort(all(p)); } // Don't Forget
```

4.4 Complex

```
typedef complex<double> complejo;
complejo a, b, c;
abs(a); // Devuelve sqrt(a.x^2 + a.y^2)
arg(a); // Devuelve el angulo entre ( -pi ; pi ]
conj(a); // Devuelve a.x - i*a.y
a*b; // Devuelve (a.x*b.x-a.y*b.y) + i*(a.x*b.y + a.y*b.x)
polar(1.0,pi); // Devuelve el complejo de modulo 1 y angulo pi
(conj(a)*b).y // Producto cruz -> a.x*b.y - a.y*b.x
(conj(b-a)*(c-a)).y/abs(b-c) // Devuelve distancia punto a -> recta bc
```

4.5 Convex Hull

```
const ld EPS = 1e-10;
void convex_hull(vector<punto> &a) {
    if(SZ(a) == 1) return;
    sort(all(a));
```

```
    punto p1 = a[0];
    vector<punto> up, down;
    up.pb(p1); down.pb(p1);
    forr(i,1,SZ(a)) {
        int n = SZ(up), m = SZ(down);
        while(n > 1 && ((up[n-1]-up[n-2])^(a[i]-up[n-1])) >= -EPS) {
            up.pop_back(); n--;
        } up.pb(a[i]);
        while(m > 1 && ((down[m-1]-down[m-2])^(a[i]-down[m-1])) <= EPS) {
            down.pop_back(); m--;
        } down.pb(a[i]);
    } // Cambiar EPS a 0 para mejor precision en enteros.
    a.clear();
    for(punto u : up) a.pb(u);
    for(int i = SZ(down)-2; i > 0; i--) a.pb(down[i]);
}
```

4.6 Formulas

```
// Shoelace formula
tipo area(vector <punto> &v) {
    tipo ans = 0.0; int n = v.size();
    forn(i,n) ans += v[i] ^ v[(i+1)%n];
    return fabs(ans/2.0);
}
tipo dist_point_line(punto &p, recta &r) {
    punto p1 = r.p, p2 = r.p + r.v;
    return fabs((p1-p)^(p2-p))/r.v.mod();
}
punto project(punto a, punto b) { //Proyeccion de b sobre a
    return ((a*b)/a.mod2()) * a;
}
tipo find_alpha(recta r, punto p) {
    return r.v.x != 0 ? (p.x-r.p.x)/r.v.x : (p.y-r.p.y)/r.v.y;
}
```

4.7 Intersection All

```
struct recta { // Puede usarse para segmentos ([p,p+v] o alpha = [0,1])
    punto v, p; // v -> director, p -> punto por donde pasa
    recta(punto p1, punto p2) { v = (p2-p1); p = p1;}
    recta() {}
    recta(tipo A, tipo B, tipo C) { // Transform Ax + By + C = 0
        v = {-B,A}; A != 0 ? p = {-C / A,0} : p = {0,-C / B};
    }
    bool is_in(punto q){return fabs((q.x-p.x)*v.y - (q.y-p.y)*v.x) < EPS;}
    punto eval(double x) {return x * v + p;}
};
bool inter_recta(recta &r1, recta &r2, punto &ans) {
    // Retorna false si son paralelas, sino guarda el punto en ans
    if(abs(r1.v ^ r2.v) < EPS) return false;
    tipo alpha = tipo((r2.p - r1.p)^r2.v) / tipo(r1.v^r2.v);
    ans = r1.p + alpha * r1.v;
    return true;
}
bool inter_seg(recta &r1, recta &r2, punto &ans) {
    if(r1.p == r2.p || r1.p == r2.p+r2.v) {ans = r1.p; return true;}
    if(r1.p+r1.v == r2.p || r1.p+r1.v == r2.p+r2.v) {
        ans = r1.p+r1.v; return true; } //Casos que coincidan extremos
    if(abs(r1.v ^ r2.v) < EPS) return false; // son paralelos
    tipo alpha = tipo((r2.p - r1.p)^r2.v) / tipo(r1.v^r2.v);
    tipo beta = tipo((r1.p - r2.p)^r1.v) / tipo(r2.v^r1.v);
    if(alpha < -EPS || beta < -EPS) return false;
    if(alpha > 1.0+EPS || beta > 1.0+EPS) return false;
    ans = r1.p + alpha * r1.v; return true;
}
struct circ { punto c; tipo r; };
tipo dist_point_line(punto &p, recta &r) {
    punto p1 = r.p, p2 = r.p + r.v;
    return fabs((p1-p)^(p2-p))/r.v.mod();
}
punto project(punto a, punto b) { //Proyeccion de b sobre a
    return ((a*b)/a.mod2()) * a;
}
vector<punto> inter_circ_line(recta r, circ c) {
    vector <punto> ans; tipo dist = dist_point_line(c.c,r);
    if(dist > c.r+EPS) return ans;
    (c.c-r.p) * r.v != 0 ? r.p = r.p : r.p = r.p + r.v;
    punto aux = c.c - r.p, dir = project(r.v,aux);
    if(fabs(dist-c.r) <= EPS) {ans.pb(r.p + dir); return ans;}
    tipo factor = sqrt(c.r*c.r - dist*dist)/dir.mod();
    ans.pb(r.p + dir + factor * dir); ans.pb(r.p + dir - factor * dir);
    return ans;
}
tipo intersection_area(circ a, circ b) {
    punto aux = (b.c - a.c); tipo dist=aux.mod(), dist2=aux.mod2();
    if(a.r + b.r - dist < -EPS) return 0;
    if(fabs(a.r - b.r) - dist > -EPS) {
        return min(a.r, b.r) * min(a.r, b.r) * 2*acosl(0); }
    tipo alpha = acosl((dist2 + a.r*a.r - b.r*b.r) / (2 * dist * a.r));
    tipo beta = acosl((dist2 + b.r*b.r - a.r*a.r) / (2 * dist * b.r));
    tipo ans1 = (alpha - sinl(alpha+alpha)*0.5) * a.r * a.r;
    tipo ans2 = (beta - sinl(beta+beta)*0.5) * b.r * b.r;
    return ans1 + ans2;
```

```
}
vector<punto> inter_circ_circ(circ a, circ b) {
    vector <punto> ans;
    if (a.c == b.c) {
        return abs(a.r-b.r) <= EPS ? vector<punto>{a.c,a.c,a.c} : ans;
    }
    b.c = b.c - a.c; punto aux = a.c; a.c = a.c - a.c;
    recta r(-2*b.c.x, -2*b.c.y, a.r*a.r - b.r*b.r + b.c.x*b.c.x + b.c.y*b.c.y);
    ans = inter_circ_line(r,a);
    forn(i,ans.size()) ans[i] = ans[i] + aux;
        return ans;
}
void tangents(punto c, double r1, double r2, vector<recta> & ans) {
    double r = r2 - r1;
    double z = c.xc.x + c.yc.y;
    double d = z - rr;
    if (d < -EPS)  return;
    d = sqrt (abs (d));
    recta l((c.x r + c.y * d) / z, (c.y * r - c.x * d) / z, r1);
    ans.push_back (l);
}
vector<recta> tangents(circ a, circ b) {
    vector<recta> ans;
    for (int i=-1; i<=1; i+=2) for (int j=-1; j<=1; j+=2) tangents (b.c-a.c, a.ri, b.rj, ans);
    forn(i,SZ(ans)) ans[i].p =ans[i].p + a.c;
    return ans;
}
```

4.8 Point In Poly

```
bool point_in_poly(vector <punto> &v, punto p) {
    // O(n) for convex
    unsigned i, j, mi, mj, c = 0;
    for(i = 0, j = v.size()-1; i < v.size(); j = i++) {
        if((v[i].y <= p.y && p.y < v[j].y) || (v[j].y <= p.y && p.y < v[i].y)) {
            mi = i; mj = j; if(v[mi].y > v[mj].y) swap(mi,mj);
            if(((p-v[mi]) ^ (v[mj]-v[mi])) < 0 ) c ^= 1;
        }
    }
    return c;
}
bool point_in_poly_convex(vector <punto> &v, punto p) { // O(log n)
    bool ans = true;
    if( ((v[1]-v[0]) ^ (v[2]-v[1])) >= 0) reverse(all(v));
    int a = 1, b = v.size()-1;
    while(b-a > 1) {
        int med = (a+b)/2;
        if( ((v[med]-v[0]) ^ (p-v[med])) <= 0 ) a = med;
        else b = med;
    }
    if( ((v[a]-v[0]) ^ (p - v[a])) > 0) ans = false;
    if( ((v[b]-v[a]) ^ (p - v[b])) > 0) ans = false;
    if( ((v[0]-v[b]) ^ (p - v[0])) > 0) ans = false;
    return ans;
}
```

4.9 Template Punto

```
typedef long double tipo; //Cambiar a long long para operar en enteros
double EPS = (double)(1e-10);
struct punto { // Puede usarse para vectores
    tipo x, y;
    punto const operator -(const punto &p) const {return {x-p.x,y-p.y};}
    punto const operator +(const punto &p) const {return {x+p.x,y+p.y};}
    tipo operator *(const punto &p) const {return x*p.x + y*p.y;}
    tipo operator ^(const punto &p) const {return x*p.y - y*p.x;}
    bool operator == (const punto &p) const {
        return (abs(x-p.x) < EPS && abs(y-p.y) < EPS); // Para double
    }
    bool operator <(punto p) const {return x != p.x ? x < p.x : y < p.y;}
    tipo arg() {return atan2(y,x);}
    tipo mod() {return sqrtl(x*x + y*y);}
    tipo mod2() {return x*x + y*y;}
};
punto operator*(tipo k, const punto &p) {return {k*p.x, k*p.y};}
```

5 Grafos

5.1 2Sat

```
// 2sat (2-satisfiability) - O(n + m) (Korasaju)
struct two_sat { // 2*x representa a x, y 2*x+1 a ~x
    int tot;
    vector<vector<int>> g, g_trans;
    vb used, assignment;
    vector<int> order, comp;
    two_sat(int _tot): tot(_tot){
        g.resize(tot); g_trans.resize(tot);
    } // tot = total de nodos (normales + negados), en general n = tot/2
    void dfs1(int v) {
        used[v] = true;
        for(auto u: g[v]) if(!used[u]) dfs1(u);
        order.pb(v);
    }
    void dfs2(int v, int cl) { // Korasaju para encontrar las SCC
        comp[v] = cl;
        for(auto u: g_trans[v]) if(comp[u] == -1) dfs2(u, cl);
    }
    bool solve() {
        order.clear(); used.assign(tot, false);
        comp.assign(tot, -1);
```

```cpp
        forn(i, tot) if(!used[i]) dfs1(i);
        int comp_act = 0;
        forn(i, tot){
            auto v = order[tot-i-1];
            if(comp[v] == -1) dfs2(v, comp_act++);
        }
        assignment.assign(tot/2, false);
        forn(i, tot/2){
            if(comp[2*i] == comp[2*i + 1]) return false;
            assignment[i] = comp[2*i] > comp[2*i+1]; // asignacion greedy de variables
        }
        return true;
    }
    void add_edge(int from, int to){ // implicancia comun from->to
        g[from].pb(to);
        g_trans[to].pb(from); // agregar en el TRANSPUESTO
    }
    void add_or(int v1, int v2) { // agrega (v1 or v2)
        add_edge(v1 ^ 1, v2); // ~v1 -> v2
        add_edge(v2 ^ 1, v1); // ~v2 -> v1
    }
    // setear variable x en true/false: add_or(x, x)/add_or(~x, ~x)
};
```

5.2 Bellman Ford

```cpp
// Time: O(VE)
typedef long long tipo;
const int MAXN = 3000;
tipo INF = (tipo)(1e18+7);
struct arista {
    int x, y; tipo w; // Edge from x to y, w = weight
};
struct nodo {
    int p; tipo d; //f -> parent, d -> distance
};
vector<nodo> ans(MAXN);
vector<int> ciclo;
bool bFord(vector<arista> &lista, int n, int start) {
    forn(i,n) ans[i].p = i, ans[i].d = INF;
    ans[start].d = 0; int x;
    forn(i,n) {
        x = -1;
        for(arista u : lista) {
            if(ans[u.y].d > ans[u.x].d + u.w) {
                ans[u.y].d = ans[u.x].d + u.w;
                ans[u.y].p = u.x;
                x = u.y;
            }
        }
    }
    if(x == -1) return false;
    else {
        forn(i,n) x = ans[x].p;
        for(int v = x ;; v = ans[v].p) {
            ciclo.push_back(v);
            if(v == x && ciclo.size() > 1) break;
        }
        reverse(all(ciclo));
        return true;
    }
}
```

5.3 Biconnected

```cpp
struct edge{ ll u, v, id; };
struct graph{
    int n;
    vector<vii> adj; // (to, id)
    vector<edge> edges;
    graph(int _n) : n(_n), adj(_n) {}
    void add_edge(ll u, ll v){
    ll id = SZ(edges);
        adj[u].pb({v, id}); adj[v].pb({u, id});
        edges.pb({u, v, id});
    }
    int add_node(){ adj.pb({}); return n++; }
    vii& operator[](ll u) { return adj[u]; }
};
struct BCC{
    int n; graph adj;
    vector<vi> comps;
    vi num, low, art, stk, bridge;
    BCC(graph &_adj): n(_adj.n), adj(_adj){
        num.resize(n), low.resize(n), art.resize(n), bridge.resize(SZ(adj.edges));
        for (ll u = 0, t; u < n; ++u) if (!num[u]) dfs(u, -1, t = 0);
    }
    void dfs(ll v, ll p, ll &t){
        num[v] = low[v] = ++t;
        stk.pb(v);
        for(auto [u, id]: adj[v]) if (u != p){
            if (!num[u]){
                dfs(u, v, t);
                low[v] = min(low[v], low[u]);
        if(low[u] > num[v]) bridge[id] = true;
                if(low[u] >= num[v]){
                    art[v] = (num[v] > 1 || num[u] > 2);
                    comps.pb({v});
                    while (comps.back().back() != u)
                        comps.back().pb(stk.back()), stk.pop_back();
                }
            }
            else low[v] = min(low[v], num[u]);
        }
    }
    // build the block cut tree
    pair<vi, graph> build_tree(){
        graph tree(0); vi id(n);
        forn(v, n) if (art[v]) id[v] = tree.add_node();
        for (auto &comp : comps){
            ll node = tree.add_node();
            for(ll v: comp){
                if (!art[v]) id[v] = node;
                else tree.add_edge(node, id[v]);
```

```
            }
        }
        return {id, tree};
    }
};
```

5.4 Blossom

```
/* Encuentra un matching en un grafo no bipartito
 * en O(nm) mediante el algoritmo de Edmond's blossom
 * Los matchings estan en (i, mate[i]), con i < mate[i]
 */
vi Blossom(vector<vi> &g) {
  int n = SZ(g), timer = -1;
  vi mate(n, -1), label(n), parent(n),
     orig(n), aux(n, -1), q;
  auto lca = [&](int x, int y) {
    for (timer++; ; swap(x, y)) {
      if (x == -1) continue;
      if (aux[x] == timer) return x;
      aux[x] = timer;
      x = (mate[x] == -1 ? -1 : orig[parent[mate[x]]]);
    }
  };
  auto blossom = [&](int v, int w, int a) {
    while (orig[v] != a) {
      parent[v] = w; w = mate[v];
      if (label[w] == 1) label[w] = 0, q.pb(w);
      orig[v] = orig[w] = a; v = parent[w];
    }
  };
  auto augment = [&](int v) {
    while (v != -1) {
      int pv = parent[v], nv = mate[pv];
      mate[v] = pv; mate[pv] = v; v = nv;
    }
  };
  auto bfs = [&](int root) {
    fill(all(label), -1);
    iota(all(orig), 0);
    q.clear();
    label[root] = 0; q.pb(root);
    forn(i, SZ(q)){
      int v = q[i];
      for (auto x : g[v]) {
        if (label[x] == -1) {
          label[x] = 1; parent[x] = v;
          if (mate[x] == -1)
            return augment(x), 1;
          label[mate[x]] = 0; q.pb(mate[x]);
        } else if (label[x] == 0 && orig[v] != orig[x]) {
          int a = lca(orig[v], orig[x]);
          blossom(x, v, a); blossom(v, x, a);
        }
      }
    }
    return 0;
  };
  // Time halves if you start with (any) maximal matching.
  forn(i, n) if(mate[i] == -1) bfs(i);
  return mate;
}
```

5.5 Componentes Fuertemente Conexas

```
/*
   Algoritmo para hallar las componentes fuertmente conexas.
   Una componente es fuertemente conexa cuando para todo nodo
   perteneciente a la componente, se puede llegar a cualquier
   otro nodo tambien perteneciente a la componente.
*/
struct SCC {
    int n, scc;
    vector<vi> g, gr, ans;
    vb visto;
    vi order, comp_act, component;
    SCC(vector<vi> &_g): n(SZ(_g)) {
        g = _g;
        gr.resize(n), visto.resize(n), component.resize(n);
        forn(v, n) for(auto u: g[v]) gr[u].pb(v); // me lo creo aca el grafo traspuesto
        find_scc();
    }
    void DFS1 (int v) {
        visto[v] = true;
        for (int u : g[v]) if(!visto[u]) DFS1(u);
        order.pb(v);
    }
    void DFS2 (int v) {
        visto[v] = true;
        comp_act.pb(v);
        for (int u : gr[v]) if(!visto[u]) DFS2(u);
    }
    void find_scc() {
        fill(all(visto),false);
        forn(i,n) if(!visto[i]) DFS1(i);
        fill(all(visto),false);
        forn(i,n) {
            int v=order[n-i-1];
            if(!visto[v]) {
                DFS2(v);
                ans.pb(comp_act);
                comp_act.clear();
            }
        }
        scc = SZ(ans); // cantidad de scc's
        forn(i, scc) for(auto v: ans[i]) component[v] = i;
    }
};
```

5.6 Dinic

```
// Dinic: Max Flow en O(V^2 E). Para el grafo bipartito con source
// y sink dummy, funciona en O(sqrt(V) E). Equivalente a Hopcroft-Karp.
// Matching: aristas saturadas (que no incluyan source/sink)
```

```cpp
// Min cut: nodos con dist>=0 vs nodos con dist<0
// MVC: Nodos izquierda con dist<0 + nodos derecha con dist>0
// Maximum Independent Set: complemento de MVC (N-MVC)
struct Dinic{
    int nodes,src,dst;
    vector<int> dist,q,work;
    struct edge { int to,rev; ll f,cap; };
    vector<vector<edge>> g;
    Dinic(int x):nodes(x),dist(x),q(x),work(x),g(x){}
    void add_edge(int s, int t, ll cap){
        g[s].pb({t,SZ(g[t]),0,cap});
        g[t].pb({s,SZ(g[s])-1,0,0});
    }
    bool dinic_bfs(){
        fill(all(dist),-1);dist[src]=0;
        int qt=0;q[qt++]=src;
        forn(qh, qt){
            int u=q[qh];
            forn(i,SZ(g[u])){
                edge &e=g[u][i];int v=g[u][i].to;
                if(dist[v]<0&&e.f<e.cap)dist[v]=dist[u]+1,q[qt++]=v;
            }
        }
        return dist[dst]>=0;
    }
    ll dinic_dfs(int u, ll f){
        if(u==dst)return f;
        for(int &i=work[u];i<SZ(g[u]);i++){
            edge &e=g[u][i];
            if(e.cap<=e.f)continue;
            int v=e.to;
            if(dist[v]==dist[u]+1){
                ll df=dinic_dfs(v,min(f,e.cap-e.f));
                if(df>0){e.f+=df;g[v][e.rev].f-=df;return df;}
            }
        }
        return 0;
    }
    ll max_flow(int _src, int _dst){
        src=_src;dst=_dst;
        ll result=0;
        while(dinic_bfs()){
            fill(all(work),0);
            while(ll delta=dinic_dfs(src,INF))
                result+=delta;
        }
        return result;
    }
};
```

5.7 Dynamic Connectivity

```cpp
// Imprime la cantidad de componentes conexas
// Resuelve el problema offline, para agregar una query es necesario
// agregar DynCon.query() en el instante que se quiera conocer la
// respuesta.
struct UnionFind {
    int n,comp;
    vector<int> uf,si,c;
    UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
        forr(i,0,n)uf[i]=i;}
    int find(int x){return x==uf[x]?x:find(uf[x]);}
    bool join(int x, int y){
        if((x=find(x))==(y=find(y)))return false;
        if(si[x]<si[y])swap(x,y);
        si[x]+=si[y];uf[y]=x;comp--;c.pb(y);
        return true;
    }
    int snap(){return c.size();}
    void rollback(int snap){
        while(c.size()>snap){
            int x=c.back();c.pop_back();
            si[uf[x]]-=si[x];uf[x]=x;comp++;
        }
    }
};
enum {ADD,DEL,QUERY};
struct Query {int type,x,y;};
struct DynCon {
    vector<Query> q;
    UnionFind dsu;
    vector<int> mt;
    map<pair<int,int>,int> last;
    DynCon(int n):dsu(n){}
    void add(int x, int y){
        if(x>y)swap(x,y);
        q.pb((Query){ADD,x,y});mt.pb(-1);last[{x,y}]=q.size()-1;
    }
    void remove(int x, int y){
        if(x>y)swap(x,y);
        q.pb((Query){DEL,x,y});
        int pr=last[{x,y}];mt[pr]=q.size()-1;mt.pb(pr);
    }
    void query(){q.pb((Query){QUERY,-1,-1});mt.pb(-1);}
    void process(){ // answers all queries in order
        if(!q.size())return;
        forr(i,0,q.size())if(q[i].type==ADD&&mt[i]<0)mt[i]=q.size();
        go(0,q.size());
    }
    void go(int s, int e){
        if(s+1==e){
            if(q[s].type==QUERY) // answer query using DSU
                cout << dsu.comp << " ";
            return;
        }
        int k=dsu.snap(),m=(s+e)/2;
        for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s)dsu.join(q[i].x,q[i].y);
        go(s,m);dsu.rollback(k);
        for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join(q[i].x,q[i].y);
        go(m,e);dsu.rollback(k);
    }
};
```

5.8 Eulerian Cycle

```
// Todos tienen que cumplir: SZ(path) == tot_edges+1
//
// Existencia directed:
// Path: Que el start tenga out_deg=in_deg+1
//       Que el final tenga in_deg=out_deg+1
//       Que el resto tenga in_deg=out_deg
//
// Cycle: Todos los nodos in_deg=out_deg
//
// Existencia undirected:
// Path: Que el start y final tengan deg impar, el resto todo par
// Cycle: Todos los nodos in_deg=out_deg
// Directed version (uncomment commented code for undirected)
struct edge {
  int y;
//  list<edge>::iterator rev;
  edge(int _y):y(_y){}
};
list<edge> g[MAXN];
void add_edge(int a, int b){
  g[a].push_front(edge(b));//auto ia=g[a].begin();
//  g[b].push_front(edge(a));auto ib=g[b].begin();
//  ia->rev=ib;ib->rev=ia;
}
vector<int> p;
void go(int x){
  while(SZ(g[x])){
    int y=g[x].front().y;
    //g[y].erase(g[x].front().rev);
    g[x].pop_front();
    go(y);
  }
  p.pb(x);
}
vector<int> get_path(int x){ // get a path that begins in x
// check that a path exists from x before calling to get_path!
  p.clear();go(x);reverse(all(p));
  return p;
}
```

5.9 Floyd Warshall

```
//~ Minino camino entre todos los nodos
//~ O(n^3) - Tambien detecta ciclos negativos
void floyd(vector<vector<int>>&matriz, ll n) {
    forn(k,n) forn(i,n) forn(j,n) {
        matriz[i][j]=min(matriz[i][j],matriz[i][k]+matriz[k][j]);
    }
}
```

5.10 Fordfulkerson

```
struct FordFulkerson {
  // Time complexity ((V+E) * MAXFLOW)
  vector < vector<arista> > g;
  map < pair<int,int>,int > cap;
  FordFulkerson(int n) {
    g.resize(n+5);
  }
  void add_edge(int x, int y, int z) { // x --> y con cap z
    g[x].pb({y});
    g[y].pb({x});
    cap[{x,y}] = z; // cap[{x,y}] += z; if multiedges allowed
  }
  void DFS(int cur, vector <int> &parent, vector<int> &flow, int prev, int cur_flow = INF) {
    parent[cur] = prev;
    flow[cur] = cur_flow;
    for(auto [next] : g[cur]) {
      if(parent[next] != -1 or cap[{cur,next}] == 0) continue;
      DFS(next,parent,flow,cur,min(cur_flow,cap[{cur,next}]));
    }
  }
  int max_flow(int s, int t) {
    int maxflow = 0;
    vector <int> parent(MAXN,-1), flow(MAXN,-1);
    DFS(s,parent,flow,s);
    //~ forn(i,t) cerr << flow[i] << " "; cerr << endl;
    while(flow[t] > 0) {
      int new_flow = flow[t];
      maxflow += new_flow;
      int cur = t;
      while(cur != s) {
        int prev = parent[cur];
        cap[{prev,cur}] -= new_flow;
        cap[{cur,prev}] += new_flow;
        cur = prev;
      }
      fill(all(parent),-1);
      fill(all(flow),0);
      DFS(s,parent,flow,s);
    }
    return maxflow;
  }
};
```

5.11 Hungarian

```
/* Algoritmo O(n^3) para assignment problem
 * Inspirando el el notebook vasito, quien se inspiro
 * en http://e-maxx.ru/algo/assignment_hungary
 */
typedef long double td; typedef vector<td> vd;
const td INF=1e100;//for maximum set INF to 0, and negate costs
bool zero(td x){return fabs(x)<1e-9;}//change to x==0, for ints/ll
struct Hungarian{
    int n; vector<vd> cs; vi L, R;
    Hungarian(int N, int M):n(max(N,M)),cs(n,vd(n)),L(n),R(n){
        forn(x,N)forn(y,M)cs[x][y]=INF;
    }
    void set(int x,int y,td c){cs[x][y]=c;}
    td assign() {
```

```
        int mat = 0; vd ds(n), u(n), v(n); vi dad(n), sn(n);
        forn(i,n)u[i]=*min_element(all(cs[i]));
        forn(j,n){v[j]=cs[0][j]-u[0];forr(i,1,n)v[j]=min(v[j],cs[i][j]-u[i]);}
        L=R=vi(n, -1);
        forn(i,n)forn(j,n)
            if(R[j]==-1&&zero(cs[i][j]-u[i]-v[j])){L[i]=j;R[j]=i;mat++;break;}
        for(;mat<n;mat++){
            int s=0, j=0, i;
            while(L[s] != -1)s++;
            fill(all(dad),-1);fill(all(sn),0);
            forn(k,n)ds[k]=cs[s][k]-u[s]-v[k];
            for(;;){
                j = -1;
                forn(k,n)if(!sn[k]&&(j==-1||ds[k]<ds[j]))j=k;
                sn[j] = 1; i = R[j];
                if(i == -1) break;
                forn(k,n)if(!sn[k]){
                    auto new_ds=ds[j]+cs[i][k]-u[i]-v[k];
                    if(ds[k] > new_ds){ds[k]=new_ds;dad[k]=j;}
                }
            }
            forn(k,n)if(k!=j&&sn[k]){auto w=ds[k]-ds[j];v[k]+=w,u[R[k]]-=w;}
            u[s] += ds[j];
            while(dad[j]>=0){int d = dad[j];R[j]=R[d];L[R[j]]=j;j=d;}
            R[j]=s;L[s]=j;
        }
        td value=0;forn(i,n)value+=cs[i][L[i]];
        return value;
    }
};
```

5.12 Max Flow Min Cost Multiedge

```
typedef long long tipo;
typedef long long tipo_cost;
const int MAXN = 505;
tipo INF = (tipo)(1e9+7);
tipo_cost MAX_COST = (tipo_cost)(1e16);
struct arista {
  int v;
  tipo cap;
  tipo_cost c; // Cost
  int id;
};
struct prev_node {
  int prev, id;
  void setting() {prev = -1, id = -1;}
};
vector < vector<arista> > g(MAXN);
vector <tipo> cap;
void add_edge(int x, int y, tipo z, tipo_cost c) {
  int id = cap.size();
  g[x].pb({y,z,c,id});
  g[y].pb({x,0,-1*c,id+1});
  cap.pb(z); cap.pb(0);
}
void Bellman_Ford(int s, int t, vector <prev_node> &parent, vector <tipo_cost> &d) {
  for(prev_node u : parent) u.setting();
  fill(all(d),MAX_COST);
  vector <bool> inq(MAXN,false);
  queue <int> q; q.push(s); parent[s] = {s,-1}; inq[s] = true; d[s] = 0;
  while(q.size() > 0) {
    int u = q.front(); q.pop(); inq[u] = false;
    for(arista next : g[u]) {
      if(cap[next.id]>0 && d[next.v] > d[u] + next.c) {
        d[next.v] = d[u] + next.c;
        parent[next.v] = {u,next.id};
        if(!inq[next.v]) {
          inq[next.v] = true;
          q.push(next.v);
        }
      }
    }
  }
}
tipo_cost max_flow_min_cost(int s, int t, int total) {
  tipo flow = 0;
  tipo_cost min_cost = 0;
  vector <prev_node> parent(MAXN);
  vector <tipo_cost> d(MAXN);
  while(true) {
    Bellman_Ford(s,t,parent,d);
    if(d[t] == MAX_COST) break;
    tipo new_flow = INF;
    int cur = t;
    while(cur != s) {
      new_flow = min(new_flow,cap[parent[cur].id]);
      cur = parent[cur].prev;
    }
    flow += new_flow;
    min_cost += d[t] * (tipo_cost)(new_flow);
    cur = t;
    while(cur != s) {
      int prev = parent[cur].prev;
      int id = parent[cur].id;
      cap[id] -= new_flow;
      cap[(id^1)] += new_flow;
      cur = prev;
    }
  }
  if(flow < total) return -1;
  return min_cost;
}
```

5.13 Min Cost Flow

```
/*
Max flow min cost, trabaja con Dijkstra y funciones potenciales
en O(n^3 m).
Tolera multiedge, pero no ciclos negativos de costo (esto es complicado en gral.)
Si es necesario el min cost flow (para un flow K fijo),
agregar una sink mas t', unir t->t' con una arista de cap K y costo 0
```

```
y correr flow entre s y t'.
*/
typedef ll tf; // tipo que se usa para el flow
typedef ll tc; // tipo que se usa para el cost
const tf INFFLOW=1e9;
const tc INFCOST=1e9;
struct MCF {
  int n;
  vector<tc> prio, pot; vector<tf> curflow; vector<int> prevedge,prevnode;
  priority_queue<pair<tc, int>, vector<pair<tc, int>>, greater<pair<tc, int>>> q;
  struct edge{int to, rev; tf f, cap; tc cost;};
  vector<vector<edge>> g;
  MCF(int _n):n(_n),prio(_n),pot(_n),curflow(_n), prevedge(_n),prevnode(_n),g(_n){}
  void add_edge(int s, int t, tf cap, tc cost) {
    g[s].pb({t,SZ(g[t]),0,cap,cost});
    g[t].pb({s,SZ(g[s])-1,0,0,-cost});
  }
  pair<tf,tc> get_flow(int s, int t) {
    tf flow=0; tc flowcost=0;
    while(1){
      q.push({0, s});
      fill(all(prio),INFCOST);
      prio[s]=0; curflow[s]=INFFLOW;
      while(!q.empty()) {
        auto cur=q.top(); q.pop();
        tc d=cur.first; int u=cur.second;
        if(d!=prio[u]) continue;
        forn(i, SZ(g[u])){
          edge &e=g[u][i];
          int v=e.to;
          if(e.cap<=e.f) continue;
          tc nprio=prio[u]+e.cost+pot[u]-pot[v];
          if(prio[v]>nprio) {
            prio[v]=nprio;
            q.push({nprio, v});
            prevnode[v]=u; prevedge[v]=i;
            curflow[v]=min(curflow[u], e.cap-e.f);
          }
        }
      }
      if(prio[t]==INFCOST) break;
      forn(i, n) pot[i]+=prio[i];
      tf df=min(curflow[t], INFFLOW-flow);
      flow+=df;
      for(int v=t; v!=s; v=prevnode[v]) {
        edge &e=g[prevnode[v]][prevedge[v]];
        e.f+=df; g[v][e.rev].f-=df;
        flowcost+=df*e.cost;
      }
    }
    return {flow,flowcost};
  }
};
```

5.14 Toposort

```
// Topological sort
// Complexity: O(n+m)
vi tsort(vector<vi> &g, int n){  // lexicographically smallest topological sort
    vi r; priority_queue<ll> q;
    vi d(2*n);
    forn(i, n) for(ll j: g[i]) d[j]++;
    forn(i, n) if(!d[i]) q.push(-i);
    while(!q.empty()){
        ll x=-q.top(); q.pop(); r.pb(x);
        for(ll j: g[x]){
            d[j]--;
            if(!d[j]) q.push(-j);
        }
    }
    return r;  // if not DAG it will have less than n elements
}
```

6 Math

6.1 Bell

```
// Los numeros de Bell son utiles para calcular de cuantas maneras
// podemos subdivir el conjunto utilizando todos los elementos -> O(n^2)
// Bell numbers = {1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, ...}
vector<int> bellNumber(int n) {
    vector<vector<int>> bell(n+1,vector<int>(n+1));
    vector<int> ans(n+1,0); // Numeros de bell desde 0 hasta n
    bell[0][0] = 1;
    forr(i, 1, n+1){
        bell[i][0] = bell[i-1][i-1];
        forr(j, 1, i+1) bell[i][j] = bell[i-1][j-1] + bell[i][j-1];
    }
    forn(i, n+1) ans[i] = bell[i][0];
    return ans;
}
```

6.2 Berlekamp Massey

```
// Berlekamp-Massey algorithm
// Description: Finds the shortest linear recurrence relation that generates a given sequence.
// Time: O(N^2)
// Output: A vector of integers, where the first element is the constant term and the last element is the coefficient of x^(n-1).
vi BM(vi x){
    vi ls,cur; ll lf,ld;
    forn(i,SZ(x)){
        ll t=0;
        forn(j,SZ(cur)) t=(t+x[i-j-1]*(ll)cur[j])%MOD;
        if((t-x[i])%MOD==0) continue;
        if(!SZ(cur)){
            cur.resize(i+1); lf=i; ld=(t-x[i])%MOD;
            continue;
        }
        ll k=-(x[i]-t)*be(ld,MOD-2,MOD)%MOD;
        vi c(i-lf-1); c.pb(k);
        forr(j,0,SZ(ls))c.pb(-ls[j]*k%MOD);
        if(SZ(c)<SZ(cur))c.resize(SZ(cur));
```

```
        forr(j,0,SZ(cur))c[j]=(c[j]+cur[j])%MOD;
        if(i-lf+SZ(ls)>=SZ(cur)) ls=cur,lf=i,ld=(t-x[i])%MOD;
        cur=c;
    }
    forn(i,SZ(cur)) cur[i]=(cur[i]%MOD+MOD)%MOD;
    return cur;
}
```

6.3 Fft Mod

```
/**
Implementacion iterativa de FFT and FFTmod. O(N log N)
La precision de la FFT depende de la respuesta.
x <= 5e14 (double), x <= 1e18(long double)
donde x = max(ans[i]) para la FFT, y x = N*mod for FFTmod
**/
typedef double ld;
typedef complex<ld> cd;
void fft(vector<cd> &a){
    int n=SZ(a), L=31-__builtin_clz(n);
    vi rev(n);
    forn(i, n) rev[i]=(rev[i/2]+((i&1)<<L))/2;
    forn(i, n) if(i<rev[i]) swap(a[i],a[rev[i]]);
    static vector<cd> root(2,1);
    for(static int k=2;k<n;k*=2){
        root.resize(n);
        cd z=polar(1.0,acos(-1)/k);
        forr(i, k, 2*k) root[i]=root[i/2]*((i&1)?z:1);
    }
    for(int k=1;k<n;k*=2){
        for(int i=0;i<n;i+=2*k){
            forn(j, k){
                cd z=root[j+k]*a[i+j+k];
                a[i+j+k]=a[i+j]-z;
                a[i+j]+=z;
            }
        }
    }
}
vi mul(vi &a, vi &b){
    int sa=SZ(a), sb=SZ(b);
    if(sa==0||sb==0) return {};
    int n=1<<(32-__builtin_clz(sa+sb-2));
    vector<cd> f(n),h(n);
    forn(i, n) f[i]=cd(ld(i<sa?a[i]:0),ld(i<sb?b[i]:0));
    fft(f);
    forn(i, n){
        ll j=(-i)&(n-1);
        h[i]=(f[j]*f[j]-conj(f[i]*f[i]))*cd(0,-0.25/n);
    }
    fft(h);
    vi c(sa+sb-1);
    forn(i, sa+sb-1) c[i]=ll(round(h[i].real()));
    return c;
}
vi mul_mod(vi &a, vi &b, ll mod){
    int sa=SZ(a), sb=SZ(b);
    if(sa==0||sb==0) return {};
    vi a1(sa),a2(sa),b1(sb),b2(sb);
    forn(i, sa) a1[i]=(a[i]&((1<<15)-1)),a2[i]=(a[i]>>15);
    forn(i, sb) b1[i]=(b[i]&((1<<15)-1)),b2[i]=(b[i]>>15);
    vi c1=mul(a1,b1),c2=mul(a1,b2),c3=mul(a2,b1),c4=mul(a2,b2);
    vi c(sa+sb-1);
    forn(i, sa+sb-1) c[i]=(c1[i]+((1<<15)*(c2[i]%mod+c3[i]%mod))+((1<<30)*(c4[i]%mod)))%mod;
    return c;
}
```

6.4 Fht

```
ll c1[MAXN+9],c2[MAXN+9];  // MAXN must be power of 2 !!
void fht(ll* p, int n, bool inv){
    for(int l=1;2*l<=n;l*=2)for(int i=0;i<n;i+=2*l)forn(j,l){
        ll u=p[i+j],v=p[i+l+j];
        if(!inv)p[i+j]=u+v,p[i+l+j]=u-v; // XOR
        else p[i+j]=(u+v)/2,p[i+l+j]=(u-v)/2;
        //if(!inv)p[i+j]=v,p[i+l+j]=u+v; // AND
        //else p[i+j]=-u+v,p[i+l+j]=u;
        //if(!inv)p[i+j]=u+v,p[i+l+j]=u; // OR
        //else p[i+j]=v,p[i+l+j]=u-v;
    }
}
// like polynomial multiplication, but XORing exponents
// instead of adding them (also ANDing, ORing)
vi multiply(vi &p1, vi &p2){
    int n=1<<(32-__builtin_clz(max(SZ(p1),SZ(p2))-1));
    forn(i,n)c1[i]=0,c2[i]=0;
    forn(i,SZ(p1))c1[i]=p1[i];
    forn(i,SZ(p2))c2[i]=p2[i];
    fht(c1,n,false);fht(c2,n,false);
    forn(i,n)c1[i]*=c2[i];
    fht(c1,n,true);
    return vi(c1,c1+n);
}
```

6.5 Gauss Sistema Ecuaciones

```
//~ const int INF = 2; // it doesn't actually have to be infinity or a big number
/*
Sistema de ecuaciones lineales modulares. Resuelve Ax=B.
En las filas estan las ecuaciones y en las columnas las variables.
En ans queda la solucion en caso de que sea unica. La matriz A es de n x m.
Esta matriz tiene n ecuaciones y m-1 variables, la ultima columna es la matrix b.
Complejidad O(min(n, m)nm). Si n == m, es O(n^3).
*/
ll mod(ll x) {x%=MOD; if(x<0) x+=MOD; return x;}
ll add(ll a, ll b){return mod(a+b);}
ll sub(ll a, ll b){return mod(a-b);}
ll mul(ll a, ll b){return mod(mod(a)*mod(b));}
ll be(ll a, ll p) {
  ll ans=1; for(; p; p/=2, a = mul(a,a)) if(p&1) ans=mul(ans,a);
```

```cpp
  return ans;
}
ll divi(ll a, ll b){return mul(a,be(b,MOD-2));}
int gauss(vector<vi> &a, vi &ans) {
    int n = SZ(a), m = SZ(a[0])-1;
    vector<int> where(m, -1);
    for(int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        forr(i, row, n){ // if(abs(a[i][col]) > abs(a[sel][col]))
            if(a[i][col] > a[sel][col]) sel = int(i);
        }
        if(a[sel][col] == 0) continue; // if(abs(a[sel][col]) < EPS)
        forr(i, col, m+1) swap(a[sel][i], a[row][i]);
        where[col] = row;
        forn(i, n){
            if(i != row) {
                ll c = divi(a[i][col], a[row][col]); // double c = a[i][col] / a[row][col];
                forr(j, col, m+1) a[i][j] = sub(a[i][j], mul(a[row][j], c));
            }
        }
        ++row;
    }
    ans.assign(m, 0);
    forn(i, m) if(where[i] != -1) ans[i] = divi(a[where[i]][m], a[where[i]][i]);
    forn(i, n){
        ll sum = 0; // double sum = 0;
        forn(j, m) sum = add(sum, mul(ans[j], a[i][j]));
        if(sum - a[i][m] != 0) return 0; // if(abs(sum-a[i][m]) > EPS)
        // No hay soluciones
    }
    forn(i, m) if(where[i] == -1) return INF; // Infinitas soluciones
    return 1; // Exactamente una solucion
}
// Encuentra las bases que generan la solucion
vector<vi> find_bases(vector<vi> &A, vi &ans) {
  int n = SZ(A), m = SZ(ans);
  vi var_ind;
  forn(i,SZ(ans)) if(ans[i] == 0) var_ind.pb(i);
  vector <vi> bases(SZ(var_ind),vi(m,0));
  int r = 0;
  for(auto u : var_ind) {
    bases[r][u] = 1;
    forn(i,n) {
      forn(j,m) {
        if(A[i][j] != 0) {
          bases[r][j] = divi(-A[i][u], A[i][j]);
          break;
        }
      }
    }
    r++;
  }
  return bases;
}
```

6.6 Inverse Matrix

```cpp
// modular functions in gauss
pair<ll,vector<vi>> inverse_matrix(vector<vector<ll> > &x) {
    // returns det and the inverse of the matrix,
    // if det == 0, no inverse
    int n = SZ(x); vector <vi> I(n,vi(n,0));
    ll det = 1;
    forn(i,n) I[i][i] = 1;
    forn(i,n) {
    int pos = -1;
        forr(fila,i,n) if(x[fila][i] > 0) pos = fila; // Not null pivot
        if(pos == -1) {return {0,I};} // If pos = -1, singular matrix
        if(pos != i) {
      swap(x[i],x[pos]), swap(I[i],I[pos]);
      det = mul(det,-1);
    }
    ll pivot = x[i][i];
    det = mul(det,pivot);
        forn(j,n) {
      x[i][j] = divi(x[i][j],pivot);
      I[i][j] = divi(I[i][j],pivot);
    }
        forn(fila,n) {
            if(fila == i) continue;
            pivot = x[fila][i];
            forn(j,n) {
        x[fila][j] = sub(x[fila][j],mul(pivot,x[i][j]));
        I[fila][j] = sub(I[fila][j],mul(pivot,I[i][j]));
      }
        }
    }
    return {det,I};
}
```

6.7 Lagrange

```cpp
ll lagrange(vii a, ll x){
  /* Recibe una muestra a de n puntos (xi,yi)
     y evalua el polinomio de Lagrange de
     en el punto x desconocido */
  ll ans = 0;
  int n = SZ(a);
  forn(i, n){
    ll term = a[i].second;
    forn(j, n){
      if(j != i)
        term = term*(x-a[j].first)/(a[i].first-a[j].first);
    }
    ans += term;
  }
  return ans;
}
```

6.8 Linear Rec

```cpp
const ll LOG = 60;
// Linear Recurrence
```

```cpp
// Description: Calculates the kth term of a linear recurrence relation
// init O(n^2log) query(n^2 logk)
// input: terms: first n term; trans: transition function (calcular con BM); MOD; LOG=mxlog of k
// output calc(k): kth term mod MOD
// example: {1,1} {2,1} an=2*a_(n-1)+a_(n-2); calc(3)=3 calc(10007)=71480733
struct LinearRec {
    ll n; vi terms, trans; vector<vi> bin;
    vi add(vi &a, vi &b){
        vi res(n*2+1);
        forn(i, n+1) forn(j, n+1) res[i+j]=(res[i+j]*1LL+(ll)a[i]*b[j])%MOD;
        for(ll i=2*n; i>n; --i){
            forn(j, n) res[i-1-j]=(res[i-1-j]*1LL+(ll)res[i]*trans[j])%MOD;
            res[i]=0;
        }
        res.erase(res.begin()+n+1,res.end());
        return res;
    }
    LinearRec(vi &terms, vi &trans):terms(terms), trans(trans){
        n=SZ(trans);vi a(n+1);a[1]=1;
        bin.pb(a);
        forr(i,1,LOG)bin.pb(add(bin[i-1],bin[i-1])); // Precompute powers of a
    }
    ll calc(ll k){
        vi a(n+1);a[0]=1;
        forn(i,LOG) if((k>>i)&1)a=add(a,bin[i]);
        ll ret=0;
        forn(i,n) ret=(ret+a[i+1]*terms[i])%MOD;
        return ret;
    }
};
```

6.9 Matrix Fast Pow

```cpp
typedef vector<vector<ll>> Matrix;
Matrix ones(int n) {
    Matrix r(n,vector<ll>(n));
    forn(i,n)r[i][i]=1;
    return r;
}
Matrix operator*(Matrix &a, Matrix &b) {
    int n=a.size(),m=b[0].size(),z=a[0].size();
    Matrix r(n,vector<ll>(m));
    forn(i,n)forn(j,m)forn(k,z)
        r[i][j]+=a[i][k]*b[k][j],r[i][j]%=MOD;
    return r;
}
Matrix be(Matrix b, ll e) {
    Matrix r=ones(b.size());
    while(e){if(e&1LL)r=r*b;b=b*b;e/=2;}
    return r;
}
```

6.10 Matrix Reduce

```cpp
double reduce(vector<vector<double> > &x) { // returns determinant
    int n=x.size(),m=x[0].size();
    int i=0,j=0;double r=1.;
    while(i<n&&j<m){
        int l=i;
        forr(k,i+1,n)if(abs(x[k][j])>abs(x[l][j]))l=k;
        if(abs(x[l][j])<EPS){j++;r=0.;continue;}
        if(l!=i){r=-r;swap(x[i],x[l]);}
        r*=x[i][j];
        for(int k=m-1;k>=j;k--)x[i][k]/=x[i][j];
        forr(k,0,n){
            if(k==i)continue;
            for(int l=m-1;l>=j;l--)x[k][l]-=x[k][j]*x[i][l];
        }
        i++;j++;
    }
    return r;
}
```

6.11 Print Int128

```cpp
string toString(__int128 num) {
    string str;
    do {
        int digit = num % 10;
        str = to_string(digit) + str;
        num = (num - digit) / 10;
    } while (num != 0);
    return str;
}
ostream& operator<<(std::ostream& os, __int128 t) {
    string str = toString(t);
    return os << str;
}
```

6.12 Random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
int my_random(int l, int r){ // Random number in [l, r]
    return uniform_int_distribution<int>(l, r)(rng);
}
```

6.13 Simplex

```cpp
struct Simplex {
    vector<int> X,Y;
    vector<vector<double>> A;
    vector<double> b,c;
    double z;
    int n,m;
    void add_equation(vector<double> v, double val){ A.pb(v); b.pb(val); }
  void set_objective(vector<double> _c){ c = _c; }
    void pivot(int x,int y){
        swap(X[y],Y[x]);
        b[x]/=A[x][y];
        forr(i,0,m)if(i!=y)A[x][i]/=A[x][y];
        A[x][y]=1/A[x][y];
        forr(i,0,n)if(i!=x&&abs(A[i][y])>EPS){
            b[i]-=A[i][y]*b[x];
            forr(j,0,m)if(j!=y)A[i][j]-=A[i][y]*A[x][j];
```

```
            A[i][y]=-A[i][y]*A[x][y];
        }
        z+=c[y]*b[x];
        forr(i,0,m)if(i!=y)c[i]-=c[y]*A[x][i];
        c[y]=-c[y]*A[x][y];
    }
    pair<double,vector<double>> simplex() {
    // maximiza c^T x, dado Ax<=b, x>=0
        // retorna par (maximum value, solution vector)
        n=b.size();m=c.size();z=0.;
        X=vector<int>(m);Y=vector<int>(n);
        forn(i,m)X[i]=i;
        forn(i,n)Y[i]=i+m;
        while(1){
            int x=-1,y=-1;
            double mn=-EPS;
            forn(i,n)if(b[i]<mn)mn=b[i],x=i;
            if(x<0) break;
            forr(i,m)if(A[x][i]<-EPS){y=i;break;}
            assert(y>=0 && "No solution found"); // no solution to Ax<=b
            pivot(x,y);
        }
        while(1){
            double mx=EPS;
            int x=-1,y=-1;
            forn(i,m)if(c[i]>mx)mx=c[i],y=i;
            if(y<0)break;
            double mn=1e200;
            forn(i,n)if(A[i][y]>EPS&&b[i]/A[i][y]<mn)mn=b[i]/A[i][y],x=i;
            assert(x>=0 && "c^T x is unbounded"); // c^T x is unbounded
            pivot(x,y);
        }
        vector<double> r(m);
        forn(i,n)if(Y[i]<m)r[Y[i]]=b[i];
        return mp(z,r);
    }
};
```

6.14 Simpson

```
double integrate(double f(double), double a, double b, int n=10000){
    double r=0,h=(b-a)/n,fa=f(a),fb;
    forr(i,0,n) {fb=f(a+h*(i+1));r+=fa+4*f(a+h*(i+0.5))+fb;fa=fb;}
    return r*h/6.;
}
```

7 Number Theory

7.1 Binexp Invmod

```
ll be(ll x, ll y, ll m) {
    if (y == 0) return 1;
    ll p = be(x, y/2, m) % m;
    p = (p * p) % m;
    return (y%2 == 0)? p : (x * p) % m;
}
ll inv_mod(ll x, ll m) {return be(x,m-2,m);}
```

7.2 Criba Phi Euler

```
bool is_composite[1000];
int phi[1000];
vector<long long> prime;
void sieve (int n) {
    fill(is_composite, is_composite + n, false);
    phi[1] = 1;
    for (int i = 2; i < n; ++i) {
        if (!is_composite[i]) {
            prime.push_back (i);
            phi[i] = i - 1; //i is prime
        }
        for (int j = 0; j < prime.size () && i * prime[j] < n; ++j) {
            is_composite[i * prime[j]] = true;
            if (i % prime[j] == 0) {
                phi[i * prime[j]] = phi[i] * prime[j];  //prime[j] divides i
                break;
            } else {
                phi[i * prime[j]] = phi[i] * phi[prime[j]]; //prime[j] does not divide i
            }
        }
    }
}
```

7.3 Discrete Log

```
/* Algoritmo de baby-step giant-step para
 * hallar x tal que a^x = b (mod m)
 * en O(sqrt(m))
 */
ll discrete_log(ll a, ll b, ll m) {
    a %= m, b %= m;
    ll n = sqrt(m) + 1;
    ll an = 1;
    for (ll i = 0; i < n; ++i)
        an = (an * 1ll * a) % m;
    unordered_map<ll, ll> vals;
    for (ll q = 0, cur = b; q <= n; ++q) {
        vals[cur] = q;
        cur = (cur * 1ll * a) % m;
    }
    for (ll p = 1, cur = 1; p <= n; ++p) {
        cur = (cur * 1ll * an) % m;
        if (vals.count(cur)) {
            ll ans = n * p - vals[cur];
            return ans;
        }
    }
    return -1;
}
```

7.4 Discrete Root

```
// Finds the primitive root modulo p
ll generator(ll p) {
    vi fact;
    ll phi = p-1, n = phi;
    for (ll i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            fact.push_back(i);
            while (n % i == 0)
                n /= i;
```

```
        }
    }
    if (n > 1)
        fact.push_back(n);
    forr(res, 2, p+1){
        bool ok = true;
        for (ll factor : fact) {
            if (be(res, phi / factor, p) == 1) {
                ok = false;
                break;
            }
        }
        if (ok) return res;
    }
    return -1;
}
// This program finds all numbers x such that x^k = a (mod n)
vi discrete_root(ll n, ll k, ll a){
    if (a == 0) return {0};
    ll g = generator(n);
    // Baby-step giant-step discrete logarithm algorithm
    ll sq = (ll) sqrt (n + .0) + 1;
    vector<pair<ll, ll>> dec(sq);
    forr(i, 1, sq+1)
        dec[i-1] = {be(g, i * sq * k % (n - 1), n), i};
    sort(all(dec));
    ll any_ans = -1;
    forn(i, sq){
        ll my = be(g, i * k % (n - 1), n) * a % n;
        auto it = lower_bound(all(dec), mp(my, 0));
        if (it != dec.end() && it->first == my) {
            any_ans = it->second * sq - i;
            break;
        }
    }
    if (any_ans == -1) return {};
    // Print all possible answers
    ll delta = (n-1) / __gcd(k, n-1);
    vi ans;
    for (ll cur = any_ans % delta; cur < n-1; cur += delta)
        ans.push_back(be(g, cur, n));
    sort(ans.begin(), ans.end());
    return ans;
}
```

7.5 Fast Factorization Mrabin Prho

```
typedef long double ld;
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll myrand(ll a, ll b) { return uniform_int_distribution<ll>(a, b)(rng); }
struct Factorization {
    inline ll mul(ll a, ll b, ll c){
        ll s = a * b - c * ll((ld) a / c * b + 0.5);
        return s < 0 ? s + c : s;
    }
    ll be(ll a, ll k, ll mod) {
        ll res = 1;
        for(; k; k >>= 1, a = mul(a, a, mod)) if (k & 1) res = mul(res, a, mod);
        return res;
    }
    bool miller(ll n) { // chequea si un numero es primo
        auto test = [&](ll n, ll a) {
            if (n == a) return true;
            if (n % 2 == 0) return false;
            ll d = (n - 1) >> __builtin_ctzll(n - 1);
            ll r = be(a, d, n);
            while (d < n - 1 && r != 1 && r != n - 1) {
                d <<= 1;
                r = mul(r, r, n);
            }
            return r == n - 1 || d & 1;
        };
        if(n == 2) return 1;
        for(auto p: vi{2, 3, 5, 7, 11, 13}) if(!test(n, p)) return 0;
        return 1;
    }
    ll pollard(ll n) { // devuelve un factor no trivial de n
        auto f = [&](ll x) { return ll((__int128(x) * x + 1) % n); };
        ll x = 0, y = 0, t = 30, prd = 2;
        while (t++ % 40 || __gcd(prd, n) == 1) {
            // speedup: no tomar gcd en cada iteracion
            if (x == y) x = myrand(2, n - 1), y = f(x);
            ll tmp = mul(prd, abs(x - y), n);
            if (tmp) prd = tmp;
            x = f(x), y = f(f(y));
        }
        return __gcd(prd, n);
    }
    vi factorize(ll n) {
        vi res;
        auto dfs = [&](auto &dfs, ll x) {
            if (x == 1) return;
            if (miller(x)) res.pb(x);
            else {
                ll d = pollard(x);
                dfs(dfs, d); dfs(dfs, x / d);
            }
        };
        dfs(dfs, n); sort(all(res));
        return res;
    }
};
```

7.6 Teo Chino Resto

```
// Teorema chino del resto
// Resuelve el sistema de ecuaciones modulares
// ai*xi = bi (mi)
ii extendedEuclid(ll a, ll b){ //a * x + b * y = gcd(a,b)
    ll x,y;
    if (b==0) return {1, 0};
```

```cpp
    auto p = extendedEuclid(b, a%b);
    x = p.second, y = p.first-(a/b)*x;
    if(a*x+b*y == -__gcd(a,b)) x=-x, y=-y;
    return {x, y};
}
pair<ii, ii> dioph(ll a, ll b, ll r) {
    // a*x+b*y=r donde r es multiplo de gcd(a,b);
    ll d=__gcd(a,b);
    a/=d; b/=d; r/=d;
    auto p = extendedEuclid(a,b);
    p.first*=r; p.second*=r;
    assert(a*p.first+b*p.second==r);
    return {p, {-b,a}}; // solutions: p+t*ans.second
}
ll inv(ll a, ll mod) { // inverso de a modulo mod
    assert(__gcd(a,mod)==1);
    ii sol = extendedEuclid(a,mod);
    return ((sol.first%mod)+mod)%mod;
}
ii sol(tuple<ll,ll,ll> c){ //requires inv, diophantine
    auto [a, x1, m] = c; ll d = __gcd(a,m);
    if(d==1) return {mod(x1*inv(a,m),m), m};
    return x1%d ? mp(-1LL,-1LL) : sol({a/d, x1/d, m/d});
}
// cond[i] = {ai,bi,mi} ai*xi=bi (mi); assumes lcm fits in ll
// Mucho cuidado con el overflow, usar __int128 si lcm no entra en ll
pair<ll,ll> crt(vector<tuple<ll,ll,ll>> cond) {
    // returns: (sol, lcm)
    ll x1=0,m1=1,x2,m2;
    for(auto t:cond){
        tie(x2,m2) = sol(t);
        if((x1-x2)%__gcd(m1,m2)) return {-1,-1};
        if(m1==m2) continue;
        ll k=dioph(m2,-m1,x1-x2).first.second, l=m1*(m2/__gcd(m1,m2));
        x1 = mod((__int128)m1*k+x1,l); m1=l;
    }
    return sol({1,x1,m1});
}
```

8 Python

8.1 Numbs.Py

```python
import random
x = random.randint(1,10) # (including 1 and 10)
choices = ["apple", "banana", "cherry"]
x = random.choice(choices) # Returns random element from list
x = 5 % -2   # -1 (because 5 - (-3 * -2) = 5 - 6 = -1)
x = -5 % 2   # 1 (because -5 - (-3 * 2) = -5 - (-6) = 1)
l = list() # Defining empty list (l = [])
l.extend([2,3]) # Extending an existing list
l.insert(1, 100) # Inserting at specific index
x = l.pop()
x = l.pop(2) # Removing from specific index
x = l + [4,5] # Merging two lists (does not change existing list)
l.remove(2) # Removing first occurance of an element
l = [0,1,2,3,4,5,6]
l1 = l[2:4]  # Getting sublist, returns [2,3]
l1 = l[:] # Quickly make a copy of list
l[0], l[2] = l[2], l[0] # Swapping two elements
l = list({"a"})  # Initializing, from set -> ["a"]
d = {"a": 1, "c": 2}
l = list(d) # From dict keys -> ["a", "c"]
l = list(d.keys()) # From dict keys -> ["a", "c"]
l = list(d.values()) # From dict values -> [1,2]
l = list(d.items()) # From dict key-values -> [("a", 1), ("c", 2)]
l = [1]*n # List with 1 repeated n times
l = list(range(10)) # Returns [0,1,2,3..,8,9]
l = [x+1 for x in nums] # Preparing list from another iterable
nums = [5,1,4,9,8]
for i, n in enumerate(nums): # Iterating with index
    print(f"Index: {i}, number: {n}")
for i in range(0,n,2): # Iterating over range with step size = 2
    print(i)
for i in range(len(nums)): # Option 1
    print(nums[~i])
for i in range(len(nums), -1, -1): # Option 2
    print(nums[i])
for i in range(len(nums)//2): # Two pointer (left and right end) iteration
    print(i) # 0,1,2
    print(~i) # -1 (i.e. 4), -2 (i.e. 3), -3(i.e. 2)
for x, y in zip(l1, l2): # Iterate over two lists
    print(x)
    print(y)
for i, (x, y) in enumerate(zip(l1,l2)):
    print(f"Index: {i}, x: {x}, y: {y}")
nums = [1,2,3,4]
for n in nums[:]: # Safely removing element from list while iterating
    if n % 2 == 0:
        nums.remove(n)
nums = [5,1,4,9,8]
nums.sort() # In-place sorting
nums.sort(reverse=True) # Descending order
new_nums = sorted(nums) # Getting new sorted list
new_nums = sorted(nums, reverse=True) # Descending order
nums = [1,-1,3,2,-3,-4]
nums.sort(key=abs) # Sort based on absolute value
nums = ["apple", "banana", "cherry"]
nums.sort(key=len) # Sort based on length
nums = [(0,1), (3,1), (1,2)]
nums.sort(key = lambda x : x[1]) # Sorts based on first element
nums = [{"age": 18, "name": "x"}, {"age": 12, "name": "y"}]
nums.sort(key = lambda x: x["age"]) # Sorts based age
from collections import Counter
c = Counter([1,2,4,1,2,5]) # Counter({1: 2, 2: 2, 4: 1, 5: 1})
for k, v in c.items():
```

```python
    print(f"Element: {k}, Frequency: {v}")
x = 1 in c # Membership checking
c[1] = 1 # Updating frequency
del c[1] # Removing character
from bisect import bisect_left, bisect_right # Equivalent to lower_bound and upper_bound
l = [1,2,2,3,4]
x = bisect_left(l, -1) # Returns 0
x = bisect_right(l, -1) # Returns 0
x = bisect_left(l, 10) # Returns 5
x = bisect_right(l, 10) # Returns 5
x = bisect_left(l, 2) # Returns 1 (index of first occurence of 2)
x = bisect_right(l, 2) # Returns 3 (1 + index of last occurence of 2)
output = all(x%2 == 0 for x in nums) # Check if list has all even numbers
output = any(x == 0 for x in nums) # Check if list has one or more zero value
is_pal = all(s[i] == s[~i] for i in range(len(s)//2) # Check if string is palindrome
from itertools import permutations, combinations
items = ["A", "B", "C"]
for c in combinations(items, 2):
    print(c) # Prints ("A", "B"), ("A", "C"), ("B", "C")
for p in permutations(items): # Iterate over all permutations
    print(p)
for p in permutations(items, 2): # Iterate over all permutations of length 2
    print(p)
d = dict() # Defining empty dict, also d = {}
d["a"] = 1 # Add/Update value against key
x = d["a"] # Raises KeyError if key is not present
x = d.get("a", None) # Returns default value if key is not present
if "a" in d: # Get a value and delete key
   x = d.pop("a")
if "a" in d: # Delete key
  del d["a"]
sorted_Keys = sorted({"x": 1, "a": 2}) # Returns ["a", "x"]
d.pop(next(iter(d))) # Pop first inserted key
from collections import defaultdict
d = defaultdict(int) # initializes non-present key with value 0
d = defaultdict(lambda : 1) # initializes non-present key with value 1
d = defaultdict(list) # initializes non-present key with empty list ([])
d = defaultdict(dict) # initializes non-present key with empty dict ({})
d = { x : x%2 == 0 for x in nums } # Dictionary comprehension : Preparing dict from another iterable
d = dict(['()', '[]', '{}']) # Returns {'(': ')', '[': ']', '{': '}'} # Create a dictionary from list/tuple of 2 length strings
t = (1, 2, 3) # Initialize tuple
t = (1,) # Make sure to add a comma to initialize single element tuple correctly # Otherwise it is treated as integer
t[0] = 4 # As tuples are immutable, this will raise a TypeError
d = {(1, 2): 'value'} # Tuple as dict keys
s = {(1, 2), (3, 4)}  # Tuple as set elements
def multi_value_func(): # Unpacking
    return 1, 2, 3
x, y, z = multi_value_func()
s = set() # Defining an empty set
s.add(1) # Adding element
size = len(s) # Getting size of set
s.update([1,2,3]) # Adding multiple elements to set
if 1 in s: # Removing already added element
  s.remove(1)
else:
  print("Removing an element that does not exist in set, throws KeyError.")
s.discard(1) # Removing element irrespective of whether it exists in set or not
if len(s) == 0: # Checking whether set is empty
  print("Set is empty")
if not s:
  print("Set is either empty or s is None")
s = set([1,2])
for i, x in enumerate(s): # Iterating over set
    print(f"Insertion Index {i} : Element : {x}")
s1 = {1,2,3}, s2 = {3,4,5}
union_set = s1 | s2 # {1,2,3,4,5}
intersection_set = s1 & s2 # {3}
diff_set = s1 - s2 # {1,2}
sym_diff_set = s1 ^ s2 # {1,2,4,5} (Just like XOR ,returns what is not present in both)
s1 <= s2 # Subset test (True)
s1 >= s2 # Superset test (False)
stack = [] # In python, List can be used as Stack .
stack.append(1) # Push
x = stack[-1] # Top / Peek
x = stack.pop() # Pop
size = len(stack) # # Getting size of stack
from collections import deque
dq = deque() # Defining an empty deque
dq.append(1) # Appending element at rear/back
dq.appendleft(1) # Appending element at front/head
x = dq.pop() # Removing and getting element from rear/back
x = dq.popleft() # Removing and getting element from front/head
size = len(dq) # Getting length/size of deque
x = dq[-1] # Accessing element at rear
x = dq[0] # Accessing element at front
x = dq[3] # Accessing element at any index
if len(dq) == 0: # Checking whether deque is empty
  print("Deque is empty")
if 1 in dq: # Looking for an element
  print("1 is part of deque")
for i, x in enumerate(dq): # Iterating over deque
    print(f"Index {i} : Element : {x}")
l1 = sorted(dq) # Sorted operation returns new list which is sorted
```

8.2 Strings.Py

```python
sls = s.lstrip() # Removes space from left end
```

```python
srs = s.rstrip() # Removes space from right end
ss = s.strip() # Removes space from both ends but not from the middle
x = "".join(["I", "am", "learning", "python"]) # Returns "Iamlearningpython"
x = "␣".join(["I", "am", "learning", "python"]) # With delimeter # Returns "I am learning python"
lower_case_s = s.lower()
upper_case_s = s.upper()
x = s.isalnum() # To check if string has all characters as either digit or alphabet
x = s.isalpha() # To check if string has all alphabets
x = s.isnum() # To check if string has all digits
x = isspace() # To check if string has all spaces
x = ord("b") # Returns 98 # Get unicode/ascii representation of character
x = s[2:4] # Return "sa" # Slicing
x = sorted(s) # Returns ['a', 's', 't', 'u', 'v']
x = sorted(s, reverse=True) # Returns ['v', 'u', 't', 's', 'a']
"a" in "utsav" # Returns True
s = "prefix"
x = s.startswith("pre") #Returns True
x = s.endswith("fix") #Returns True
s1 = "hello␣world␣programming"
subs1 = "world"
start_index = s1.find(subs1) # Returns 6
```

9 Strings

9.1 Aho Corasick

```cpp
struct vertex {
    map<char,int> next, go;
    int p,link,nextleaf;
    char pch;
    vector<int> leaf;
    vertex(int p=-1, char pch=-1,int nextleaf=-1) :
        p(p),pch(pch),link(-1),nextleaf(nextleaf) {}
};
vector<vertex> t;
vector <vector <int> > g(MAXN); // Suffix-links tree
void aho_init(){ //do not forget!!
    t.clear(); t.pb(vertex());
}
void add_string(string s, int id) {
    int v=0;
    for(char c:s) {
        if(!t[v].next.count(c)){
            t[v].next[c]=t.size();
            t.pb(vertex(v,c));
        }
        v=t[v].next[c];
    }
    t[v].leaf.pb(id);
}
int go(int v, char c);
int get_link(int v) {
    if(t[v].link < 0) {
        if(v == 0 || t[v].p == 0) t[v].link = 0;
        else t[v].link = go(get_link(t[v].p),t[v].pch);
        g[t[v].link].pb(v);
    }
    return t[v].link;
}
int go(int v, char c){
    if(!t[v].go.count(c)) {
        if(t[v].next.count(c))t[v].go[c]=t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v),c);
    }
    return t[v].go[c];
}
int init_next_leaf(int v) {
    if(v == 0) t[v].nextleaf=0;
    if(t[get_link(v)].leaf.size()) return t[v].nextleaf=get_link(v);
    else return t[v].nextleaf = t[get_link(v)].nextleaf != -1 ?
        t[get_link(v)].nextleaf :
            init_next_leaf(get_link(v));
}
void construct_links() { forn(i,t.size()) get_link(i); }
```

9.2 Hashing

```cpp
struct Hash {
    const ll P=1777771;
    const ll MOD[2] = {999727999, 1070777777};
    const ll PI[2] = {325255434, 10018302};
    vector<ll> h[2],pi[2];
    Hash(string& s){
        forn(k,2)h[k].resize(s.size()+1),pi[k].resize(s.size()+1);
        forn(k,2){
            h[k][0]=0;pi[k][0]=1;
            ll p=1;
            forr(i,1,s.size()+1){
                h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
                pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
                p=(p*P)%MOD[k];
            }
        }
    }
    ll get(ll s, ll e){ // [s, e] (s y e van de 0 a n-1)
        e++;
        ll h0=(h[0][e]-h[0][s]+MOD[0]);
        h0=(1LL*h0*pi[0][s])%MOD[0];
        ll h1=(h[1][e]-h[1][s]+MOD[1]);
        h1=(1LL*h1*pi[1][s])%MOD[1];
        return (h0<<32)|h1;
    }
};
```

9.3 Manacher

```cpp
vi d1;      //d1[i] = max odd palindrome centered on i
vi d2;      //d2[i] = max even palindrome centered on i
```

```
            //s  aabbaacaabbaa
            //d1 1111117111111
            //d2 0103010010301
void manacher(string& s){
    ll l=0,r=-1,n=SZ(s);
    d1.resize(n), d2.resize(n);
    forn(i, n){
        ll k = (i>r ? 1 : min(d1[l+r-i],r-i));
        while(i+k<n && i-k>=0 && s[i+k]==s[i-k]) k++;
        d1[i] = k--;
        if(i+k>r) l=i-k,r=i+k;
    }
    l=0; r=-1;
    forn(i, n){
        ll k = (i>r ? 0 : min(d2[l+r-i+1],r-i+1)); k++;
        while(i+k<=n && i-k>=0 && s[i+k-1]==s[i-k]) k++;
        d2[i] = --k;
        if(i+k-1>r) l=i-k, r=i+k-1;
    }
}
```

9.4 Prefix Function

```
vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    forr(i,1,n) {
        int j = pi[i-1];
        while(j > 0 && s[i] != s[j]) j = pi[j-1];
        if(s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
```

9.5 Suffix Array

```
void csort(vector<int>& sa, vector<int>& r, int k){
    int n=sa.size();
    vector<int> f(max(255,n),0),t(n);
    forr(i,0,n)f[RB(i+k)]++;
    int sum=0;
    forr(i,0,max(255,n))f[i]=(sum+=f[i])-f[i];
    forr(i,0,n)t[f[RB(sa[i]+k)]++]=sa[i];
    sa=t;
}
vector<int> suffix_array(string& s){ // O(n logn)
    s += '$';
    int n=s.size(),rank;
    vector<int> sa(n),r(n),t(n);
    forr(i,0,n)sa[i]=i,r[i]=s[i];
    for(int k=1;k<n;k*=2){
        csort(sa,r,k);csort(sa,r,0);
        t[sa[0]]=rank=0;
        forr(i,1,n){
            if(r[sa[i]]!=r[sa[i-1]]||RB(sa[i]+k)!=RB(sa[i-1]+k))rank++;
            t[sa[i]]=rank;
        }
        r=t;
        if(r[sa[n-1]]==n-1)break;
    }
    return sa;
}
vector<int> lcp_construction(string const& s, vector<int> const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++) rank[p[i]] = i;
    int k = 0;
    vector<int> lcp(n-1, 0);
    for (int i = 0; i < n; i++) {
        if(rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = p[rank[i] + 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+k]) k++;
        lcp[rank[i]] = k;
        if(k) k--;
    }
    return lcp;
}
bool substr_search(string &text, string &pal, vector <int> &sa) {
    int n = text.size(), a = -1, b = n-1;
    while(b-a > 1) {
        int med = (a+b)/2, pos = sa[med]; bool d = false;
        int tam = min((int)pal.size(),int(n-sa[med]));
        string check = text.substr(sa[med],tam);
        if(check < pal) a=med;
        else b = med;
    }
    int tam = min(int(pal.size()),int(n-sa[b]));
    string fin = text.substr(sa[b],tam);
    return fin == pal ? true : false;
}
ll count_substring(string &pal) {
    ll n = pal.size();
    vector <int> sa = suffix_array(pal);
    vector <int> lcp = lcp_construction(pal,sa);
    ll ans = n * (n+1) / 2;
    for(int u : lcp) ans -= (ll)u;
    return ans;
}
string lcsubstr(string s, string t) {
    string r = s + '#' + t + '$';
    int best = 0; int pos = 0;
    vector <int> sa = suffix_array(r);
    vector <int> lcp = lcp_construction(r, sa);
    forr(i, 1, SZ(sa)) {
        if(isInRange(sa[i - 1], 0, SZ(s)) && isInRange(sa[i], SZ(s) + 1, SZ(r) - 1)) {
            if(lcp[i-1] > best) best = lcp[i-1], pos = sa[i];
        }
        if(isInRange(sa[i], 0, SZ(s)) && isInRange(sa[i - 1], SZ(s) + 1, SZ(r) - 1)) {
            if(lcp[i-1] > best) best = lcp[i-1], pos = sa[i];
        }
    }
```

```
    return r.substr(pos,best);
    // To find de LCSubstr between n string, we can concatenate the n
    // strings and apply min_segtree in lcp ans intervals with at least
    // one suffix of each string.
}
```

9.6 Trie

```
const int K = 26;
struct Vertex {
    int next[K];
    int leaf;
    Vertex() {
        fill(begin(next), end(next), -1);
        leaf = 0;
    }
};
vector<Vertex> trie(1);
void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].leaf++;
}
bool find(string &pal) {
    int cur = 0;
    for(char u : pal) {
        cur = trie[cur].next[u-'a'];
        if(cur == -1) return false;
    }
    return true;
}
```

9.7 Z Function

```
vector<int> z_function(string s) {
    int n = (int) s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r)
            z[i] = min (r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            z[i]++;
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}
```

10 Tree

10.1 Centroid Cses

```
vector<int> g[MAXN];int n;
bool tk[MAXN];
int fat[MAXN]; // father in centroid decomposition
int szt[MAXN]; // size of subtree
int calcsz(int x, int f){
    szt[x]=1;
    for(auto y:g[x])if(y!=f&&!tk[y])szt[x]+=calcsz(y,x);
    return szt[x];
}
void cdfs(int x=0, int f=-1, int sz=-1){ // O(nlogn)
    if(sz<0)sz=calcsz(x,-1);
    for(auto y:g[x])if(!tk[y]&&szt[y]*2>=sz){
        szt[x]=0;cdfs(y,f,sz);return;
    }
    tk[x]=true;fat[x]=f;
    for(auto y:g[x])if(!tk[y])cdfs(y,x);
}
void centroid(){memset(tk,false,sizeof(tk));cdfs();}
```

10.2 Dynamic Connectivity

```
struct UnionFind {
    int n,comp;
    vi uf,si,c;
    UnionFind(int n=0):n(n),comp(n),uf(n),si(n,1){
        forn(i,n) uf[i]=(int)i;
    }
    int find(int x){return x==uf[x]?x:find(uf[x]);}
    bool join(int x, int y){
        if((x=find(x))==(y=find(y)))return false;
        if(si[x]<si[y])swap(x,y);
        si[x]+=si[y];uf[y]=x;comp--;c.pb(y);
        return true;
    }
    int snap(){return SZ(c);}
    void rollback(int snap){
        while(SZ(c)>snap){
            int x=c.back();c.pop_back();
            si[uf[x]]-=si[x];uf[x]=x;comp++;
        }
    }
};
enum {ADD,DEL,QUERY};
struct Query {int type,x,y;};
struct DynCon {
    vector<Query> q;
    UnionFind dsu;
    vector<int> mt;
    map<pair<int,int>,int> last;
    DynCon(int n):dsu(n){}
    void add(int x, int y){
        if(x>y)swap(x,y);
        q.pb({ADD,x,y});mt.pb(-1);last[{x,y}]=SZ(q)-1;
    }
    void remove(int x, int y){
        if(x>y)swap(x,y);
        q.pb({DEL,x,y});
        int pr=last[{x,y}];mt[pr]=SZ(q)-1;mt.pb(pr);
    }
    void query(int x, int y){q.pb({QUERY,x,y});mt.pb(-1);} // modificar que query se tiene
```

```
        que usar
    void process(){ // answers all queries in order
        if(!SZ(q)) return;
        forn(i,SZ(q))if(q[i].type==ADD&&mt[i]<0) mt[i]=SZ(q);
        go(0,SZ(q));
    }
    void go(int s, int e){
        if(s+1==e){
            if(q[s].type==QUERY) // answer query using DSU q[s]
                assert(false); // poner alguna operacion entre q[s].x y q[s].y
            return;
        }
        int k=dsu.snap(),m=(s+e)/2;
        for(int i=e-1;i>=m;--i)if(mt[i]>=0&&mt[i]<s)dsu.join(q[i].x,q[i].y);
        go(s,m);dsu.rollback(k);
        for(int i=m-1;i>=s;--i)if(mt[i]>=e)dsu.join(q[i].x,q[i].y);
        go(m,e);dsu.rollback(k);
    }
};
```

10.3 Hld

```
struct hld {
  #define rs(x) resize(x)
  int n;
  vector <vector<int>> g;
  vector <tipo> values, sz, heavy, parent, depth, top, r, pos;
  segtree st; // iterative is faster
  hld(int _n, vector <vector<int>> _g, vector <tipo> _v) {
    n = _n; g = _g; values = _v;
    sz.rs(n); parent.rs(n); depth.rs(n); top.rs(n); pos.rs(n);
    heavy.rs(n);
    forn(i,n) heavy[i] = i;
    dfs(0);
    get_order(0,0);
    st.build(r,r.size());
  }
  ll dfs(int v, int p = -1, int d = 0) {
    depth[v] = d; parent[v] = p;
    for(int &u : g[v]) {
      if(u == p) continue;
      ll tam = dfs(u,v,d+1);
      // move heavy node to the begining
      if(sz[u] > sz[heavy[v]]) heavy[v] = u, swap(u,g[v][0]);
      sz[v] += tam;
    }
    sz[v] += 1;
    return sz[v];
  }
  void get_order(int v, int t, int p = -1) { // first child is heavy
    pos[v] = r.size(); top[v] = t; r.pb(values[v]);
    for(int u : g[v]) {
      if(u == p) continue;
      if(u == heavy[v]) get_order(u,t,v);
      else get_order(u,u,v);
    }
  }
  void op(ll &res, int a, int b) { // Set operation
    res = max(res, st.query(pos[a],pos[b]));
    // ------- for segtree recursive use ----------
    // res = max(res, st.query(pos[a],pos[b]).ans);
  }
  tipo query_hld(int a, int b) {
    tipo res = NEUT;
    for(; top[a] != top[b]; b = parent[top[b]]) {
      if(depth[top[a]] > depth[top[b]]) swap(a, b);
      op(res,top[b],b);
    }
    if(depth[a] > depth[b]) swap(a, b);
    op(res,a,b);
    return res;
  }
  void update_hld(int p, tipo val) { st.update(pos[p],val); }
};
```

10.4 Lca

```
/*
Tiempo de build: O(nlogn)
Memoria: O(nlogn)
Tiempo por query: O(logn)
*/
const int MAXN = 200005, LOG = 20;
struct LCA {
    int n, root;
    vector<vector<int>> g;
    int jmp[MAXN][LOG], depth[MAXN]; // jmp[i][j] tiene el 2^j-esimo ancestro de i
    void lca_dfs(int x) {
        for(int u : g[x]) {
            if(u == jmp[x][0]) continue;
            jmp[u][0] = x; depth[u] = depth[x]+1;
            lca_dfs(u);
        }
    }
    LCA(int tam, vector<vector<int>> &tree, int r): n(tam),root(r),g(tree) {
        depth[root] = 0;
        memset(jmp,-1,sizeof(jmp)); jmp[root][0] = root;
        lca_dfs(root);
        forr(k, 1, LOG){ forn(i, n){
                if(jmp[i][k-1]<0) jmp[i][k] = -1;
                else jmp[i][k] = jmp[jmp[i][k-1]][k-1];
            }
        }
    }
    int lca(int x, int y){
        if(depth[x] < depth[y]) swap(x,y);
        for(int i = LOG-1; i >= 0; i--) {
            if(depth[x]-(1<<i) >= depth[y]) x =
```

```
                jmp[x][i];
        }
        if(x == y) return x;
        for(int i = LOG-1; i >= 0; i--) {
            if(jmp[x][i] != jmp[y][i]) x = jmp[x][i], y = jmp[y][i];
        }
        return jmp[x][0];
    }
    int dist(int x, int y) {
        return depth[x] + depth[y] - 2 * depth[lca(x,y)];
    }
};
```

10.5 Sm To Large

```
// Small to large techique.
struct query { // Queries to answer, v: vertex, c: color, idx: ans pos
    ll v, c, idx;
};
vi g[MAXN];
vector<query> q[MAXN]; // Queries to answer
vi ans(MAXN, -1), color; // Answer to each query
unordered_map<ll, ll> cnt[MAXN]; // struct to store the info and merge
int merge(int v, int u){
    if(SZ(cnt[v]) < SZ(cnt[u])) swap(u, v); // now v is the large one
    for(auto [x, y]: cnt[u]){
        cnt[v][x] += y;
    }
    cnt[u].clear(); // Clear the small one to mantain memory in O(n)
    return v; // return the large node
}
// Process the queries of v, v_repr is the representative of v (large
// node after merging v and its children)
void process_queries(int v, int v_repr){
    for(auto &[_v, c, i]: q[v]) {
    ans[i] = SZ(cnt[v_repr]); // Number of distinct colors
    //~ ans[i] = cnt[v_repr][c]; // Number of nodes with color c
  }
}
int dfs(int v, int p){
    int v_repr = v; // Initialize the representative of v
    cnt[v][color[v]]++; // Initialize counter
    for(auto u: g[v]){
        if(u == p) continue;
        int u_repr = dfs(u, v); // Get the representative of u
        v_repr = merge(v_repr, u_repr); // Merge u_repr into v_repr
    }
    process_queries(v, v_repr); // Offline process the queries of v
    return v_repr;
}
void solve(int m) { // number of queries
  dfs(0,-1);
  forn(i,m) cout << ans[i] << " ";
  cout << "\n";
}
```